

Arte Acessível: Uma Análise Comparativa de Algoritmos de Detecção de Obras de Arte

Marcus Ferreira¹, Iago Medeiros¹

¹Universidade Federal do Pará (UFPA)

marcus.ferreira@tucurui.ufpa.br, iagomedeiros@ufpa.br

Abstract. *Object detection using computer vision has proven to be a powerful tool in various fields, including accessibility for people with visual impairments. This work focuses on comparing convolutional neural network algorithms (YOLOv8, YOLOv11, RetinaNet, and Faster R-CNN) for detecting painted artworks, with the aim of evaluating their feasibility for mobile device applications. The results showed that the YOLOv8 and YOLOv11 models exhibited superior performance in terms of mean average precision (mAP) and recall, with efficient resource consumption during training and inference time.*

Resumo. *A detecção de objetos utilizando visão computacional tem se mostrado uma ferramenta muito poderosa em diversas áreas, incluindo a acessibilidade para pessoas com deficiência visual. Esse trabalho foca na comparação de algoritmos de redes neurais convolucionais (YOLOv8, YOLOv11, RetinaNet e Faster R-CNN) para detecção de obras pintadas, com o objetivo de avaliar sua viabilidade para aplicações de dispositivos móveis. Os resultados demonstraram que os modelos YOLOv8 e YOLOv11 apresentaram um desempenho superior em termo de precisão média (mAP) e recall, com um bom consumo de recursos durante o treinamento e tempo de inferência.*

1. Introdução

Com o constante avanço de tecnologias digitais, surgem novas ferramentas capazes de mitigar problemas que permeiam a sociedade ao longo das épocas, como as questões de acessibilidade e inclusão de pessoas que possuem algum tipo de deficiência visual. Segundo dados apresentados pelo Instituto Brasileiro de Geografia e Estatística (IBGE), realizado em 2022, aproximadamente 18,6 milhões de pessoas (8,9%) de 2 anos ou mais possuem algum tipo de deficiência no Brasil. Dentre essas, a segunda dificuldade mais relatada foi a de enxergar, mesmo utilizando óculos ou lentes de contato, afetando 3,1% dos entrevistados [IBGE 2023].

É relevante observar que, ao falarmos de pessoas com deficiência visual, estamos nos referindo a um grupo de pessoas com uma deficiência no sentido da visão. Essa deficiência deve ser interpretada em diversas categorias, já que a perda de visão é subjetiva, revelando-se de maneira diferenciada de indivíduo para indivíduo [Martins 2008]. Ao abordar o tema artes e visualidade, é importante compreender que habitamos um mundo que se apresenta de forma majoritariamente visual. Sendo assim, no campo artístico, a apreciação das obras ocorre, em grande parte, através da observação visual da mesma, pois os olhos tendem a resumir em sua totalidade o fenômeno artístico apresentado [Patel et al. 2025].

Desse modo, o ensino da arte para todos deve ser tratado como a sua devida importância, pois a mesma possui função ativa na produção de conhecimento e crescimento pessoal do indivíduo, pois negar acesso de grupos de pessoas, nomeadamente com deficiência, ao ensino da arte é desperdiçar o potencial inerente do sujeito. Portanto, a responsabilidade de mitigar a exclusão desse grupo de pessoas deve ser reconhecida pelos núcleos sociais em que o mesmo está inserido, como a escola, a família, o Estado e a própria sociedade em geral [de Milano and Honorato 2010].

Dentre as tecnologias capazes de diminuir a limitação imposta pela deficiência visual, está a visão computacional. Sendo esse o ramo da ciência responsável pela visão de uma máquina, possibilitando o computador visualizar o ambiente à sua volta e extrair informações significativas a partir de imagens capturadas por câmeras de vídeo, scanners ou qualquer outro dispositivo capaz de fornecer uma imagem como entrada [de Milano and Honorato 2010].

Sistemas que possuem a capacidade de realizar o reconhecimento de objetos a partir de uma imagem, podem ser aplicados em uma grande variedade de projetos, como o desenvolvimento de carros autônomos, controle de equipamentos via gesto, monitoramento de locais públicos e etc. Para isso, deve ser realizado o estudo e compreensão de cada aplicação para obter o melhor equilíbrio entre o desempenho da técnica e o custo computacional [Bernardo 2019]. Este trabalho visa integrar ferramentas de visão computacional e algoritmos de inteligência artificial que sejam capazes de reconhecer obras de artes pintadas, auxiliando assim pessoas com deficiência visual, promovendo maior inclusão e acessibilidade das mesmas em ambientes como museus e exposições.

2. Trabalhos relacionados

Com a constante popularização e simplificação de ferramentas que implementem recursos de visão computacional, diversos projetos buscam ampliar a acessibilidade através da detecção de objetos visuais. O trabalho de [Olivatto 2021] utiliza rede neural convolucional através da ferramenta YOLOV4, visando a identificação de rampas de acessibilidade em passeios públicos, conseguindo obter uma precisão média de 75%. No entanto, o modelo obteve dificuldades de detecção oriundos da falta de padronização das rampas de acessibilidade, especialmente em cidades com um menor número de habitantes.

[de Carvalho and do Nascimento 2025] exploram as transformações culturais e tecnológicas na era digital, em particular, analisando de forma extensiva sobre como as tecnologias assistivas estão desempenhando um papel crucial na inclusão digital e no empoderamento de indivíduos com deficiências em museus. Este estudo é especialmente relevante para o desenvolvimento de novas tecnologias assistivas, pois oferece novas perspectivas sobre a integração dessas ferramentas no contexto cultural e digital contemporâneo.

O trabalho de [Tian et al. 2010] propõe um algoritmo de detecção de portas baseado em visão computacional para auxiliar pessoas cegas a navegar em ambientes internos desconhecidos. O método utiliza um modelo geométrico de portas baseado em características estáveis, como arestas e cantos, ao invés de atributos de aparência, como cor e textura. O algoritmo demonstrou robustez em diferentes condições de iluminação, perspectiva e oclusão, alcançando uma taxa de detecção de 91,9%. No entanto, uma limitação do estudo é a dificuldade em diferenciar portas de outros objetos grandes de formato se-

melhante, como estantes e armários, devido à falta de informações de profundidade.

O trabalho de [Khan et al. 2014] foca no desenvolvimento de um conjunto de dados de imagens de pinturas de alta resolução, denominado Paintings-100, para classificação de artistas e estilos. A avaliação preliminar do conjunto de dados Paintings-100, utilizando redes neurais convolucionais (CNNs), demonstra resultados promissores na classificação de artistas, alcançando uma precisão de até 38% através da fusão de decisões de modelos que analisam tanto a imagem completa quanto patches aleatórios. Este estudo destaca a importância de conjuntos de dados de alta qualidade e diversificados para o avanço em tarefas de classificação de imagens no domínio da arte.

Todos os trabalhos apresentados demonstram a aplicação da visão computacional em diferentes domínios, abrangendo desde acessibilidade urbana até a criação de conjuntos de dados para a classificação de obras de arte. No entanto, observa-se uma lacuna na literatura no que diz respeito ao uso da visão computacional para aprimorar a experiência e promover a inclusão de pessoas com deficiência visual no contexto artístico. Este trabalho, portanto, propõe a comparação de diversos algoritmos para detecção de objetos artísticos visuais, com o objetivo de determinar a opção mais viável para futuras implementações.

3. Detector de obras de arte

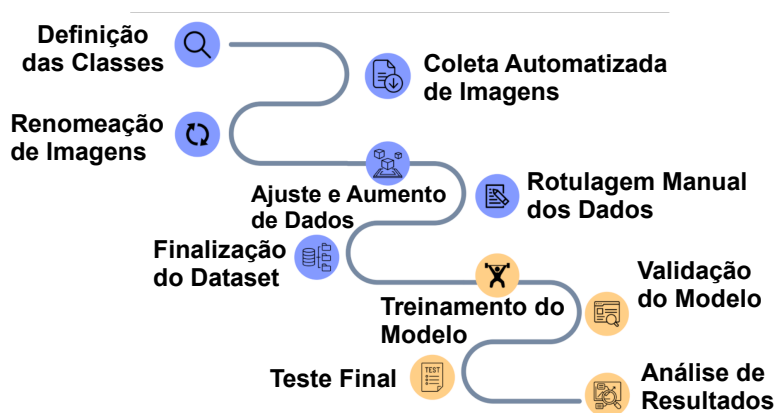


Figura 1. Etapas de trabalho

Para a concepção desse projeto, foram utilizadas diversas tecnologias que são de extrema importância nas etapas de anotação de dados, detecção de objetos e processamento de linguagem natural. Figura 1 apresenta as etapas de trabalho para a proposta apresentada, onde caracteriza-se por 10 etapas que englobam todo o processo de coleta, melhoria, treinamento, teste e análise de resultados finais.

3.1. Criação do DataSet

A criação do dataset envolveu o uso de definição de quais obras seriam usadas, ou seja, a etapa de detecção de classes. Foram selecionadas algumas obras famosas, que representam diferentes aspectos culturais da sociedade humana. A lista selecionada pode ser vista na Figura 3 e são as seguintes obras: 1. Les Noces de Cana, 2. Liberty Leading The

People, 3. Meisje Met de Parel, 4. Mona Lisa, 5. Virgin Of The Rocks, 6. The Creation Of Adam, 7. The Great Wave off Kanagawa, 8. The Night Watch e 9. The Starry Night.



Figura 2. Obras que compõem o dataset.

Para a criação do dataset com imagens das obras pintadas, foi utilizado um algoritmo desenvolvido para realizar a coleta automatizada de imagens da internet a partir do nome da obra em específico, facilitando a coleta de dados para o treinamento do modelo. Esse algoritmo feito em Python utiliza a biblioteca Bing Image Downloader e está disponível para conferência¹. Para a rotulação dos dados após a coleta, utilizou-se o software Labellmg, uma ferramenta popular de código aberto para rotulagem (renomeação de imagens), desenvolvido em Python, suportando formatos PASCAL VOC, CreateML e, principalmente, YOLO. [Tzutalin and Contributors 2024]

Durante a criação do dataset, detectou-se uma grande dificuldade na obtenção de amostras para treinamento, para contornar esse problema foi utilizado a biblioteca Albumentations em um script Python. A biblioteca destaca-se por criar novas amostras a partir das já existentes, tornando a nova imagem espelhada (*flip*), borrada (*blur*), rotacionada (*rotate*), com erros (*dropout*), mais ou menos brilhante (*brightness*) e cortada (*crop*) [Team 2024], conforme visto na Figura 3. Esta etapa de ajuste e aumento de dados serve para aumentar a oferta de conteúdo a ser treinado e avaliado. Após isso, foram feitos a rotulagem manual, pois o aumento de dados demanda novos nomes. Enfim, teve a finalização do dataset com 3529 imagens de treinamento, 770 de validação e 750 para teste.

¹<https://github.com/MarcusCarvalho21/ArteAcessivel.git>

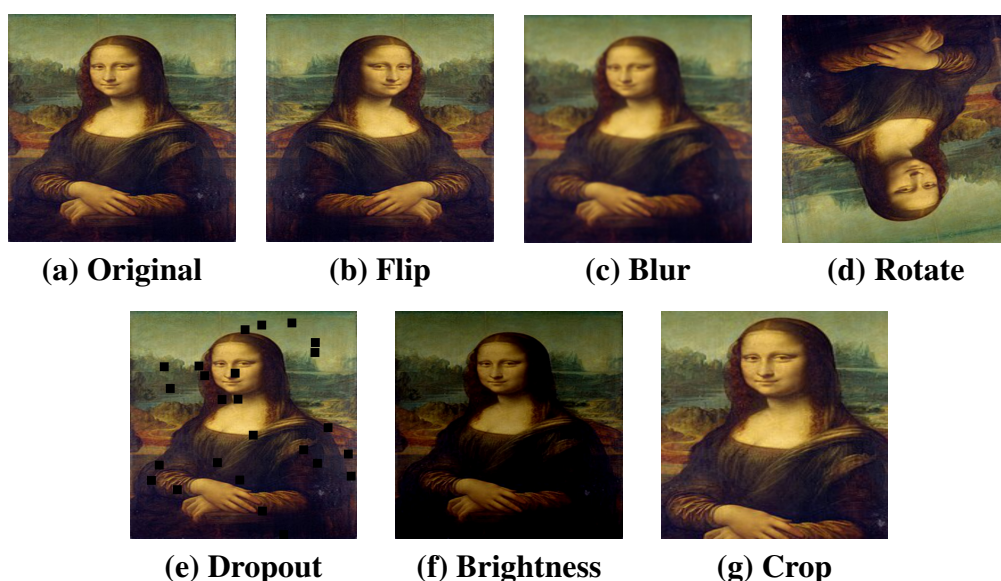


Figura 3. Imagens alteradas para aumento de dados e construção mais desafiadora do *dataset*.

3.2. Algoritmos avaliados de Redes Neurais Convolucionais

Após a criação do dataset, é necessário realizar a avaliação dos algoritmos mais adequados para o detector de obras de arte. Para isso, é preciso realizar o treinamento dos diversos modelos analisados, sua validação de modelo, além dos testes finais e análise destes resultados. Neste trabalho, foi utilizado três algoritmos famosos e distintos, chamados de YOLO, RetinaNet e Faster R-CNN, onde todos são extensões de Redes Neurais Convolucionais (CNN).

O YOLO (You Only Look Once) é um modelo de detecção de objetos e segmentação de imagens. Diferente de outras abordagens que utilizam R-CNN, o YOLO surge como uma ferramenta simplificada, já que o mesmo trata a detecção de objetos como um problema de regressão, partindo dos pixels da imagem para as coordenadas das caixas delimitadoras e as probabilidades de classe [Redmon et al. 2016].

O RetinaNet é um modelo de detecção de objetos que busca o equilíbrio entre precisão e eficiência computacional. O modelo é composto por três componente principais, sendo eles, o backbone denominado Feature Pyramid Network (que aprimora uma rede convolucional padrão de forma eficiente e multiescalar), uma sub-rede de classificação e, por fim, uma sub-rede de regressão [Lin et al. 2017].

Faster R-CNN foi desenvolvido para lidar com a detecção de objetos, visando atacar limitações apresentados na algoritmo R-CNN. O seu parâmetro de entrada é uma imagem e a partir da mesma é gerada caixas delimitadoras, onde cada uma conta com um rótulo de classe de objetos, onde o algoritmo trabalha com duas grande etapas de trabalho, inicialmente é gerado um conjunto de propostas, para assim, realizar a classificação final [Boesch 2024].

4. Resultados

Nesta seção, apresentamos os resultados dos experimentos realizados com os diferentes modelos de detecção de objetos, focando no desempenho do YOLO e em sua comparação

com outros algoritmos de redes neurais, como Retinanet e Faster-RCNN. A primeira avaliação é relacionado à determinação de quais versões dos algoritmos é mais apropriada para a tarefa de detecção de pinturas, visto que eles apresentam variações (refinos e finalidades diferentes) entre as próprias técnicas, geralmente envolvendo versões completas e mais lentas, e versões menores e mais rápidas. A segunda parte da avaliação consiste no cálculo de qual é a melhor técnica geral para detecção.

4.1. Métricas avaliadas e ambiente de simulação

Para compreender melhor os resultados obtidos, é fundamental destacar as métricas utilizadas na avaliação dos modelos: tempo de treinamento, precisão, recall e precisão média. O tempo de treinamento foi medido em horas, representando o período necessário para completar o treinamento de cada modelo. Dentre as métricas avaliadas, apenas o tempo de treinamento, tempo de inferência e recursos computacionais são fatores onde é recomendado serem resultados menores, enquanto todas as outras métricas apresentam resultados melhores quando possuem valores maiores. A Equação 1 descreve a operação de cálculo de recall, que é outra métrica relevante. Recall (também chamado de Revocação) mede a proporção de objetos reais que foram corretamente detectados, utilizando para o seu cálculo valores de verdadeiros positivos (TP) e falsos negativos (FN).

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (1)$$

Precisão mede se a quantidade de predições positivas são realmente verdadeiras, onde seu calculo baseia-se no valores de verdadeiros positivos (TP) e falsos positivos (FP), conforme visto na Equação 2, que descreve a operação de cálculo da precisão.

$$\text{Precisão} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

Com base nos valores de Precisão e Revocação, é construída a curva Precisão-Revocação (PR). A Average Precision (AP) de cada classe corresponde à área sob essa curva, e o mAP é obtido pela média das APs de todas as classes, sendo descrita pela Equação 3, onde N é o número total de classes presentes no modelo.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N AP_i \quad (3)$$

Para avaliar o tempo de inferência dos modelos em um determinado conjunto de imagens, foi calculado o tempo médio de inferência utilizando a Equação 4, onde N representa o número total de imagens, T_i o tempo de inferência da i -ésima imagem e $T_{\text{médio}}$ o tempo médio de inferência por imagem.

$$T_{\text{médio}} = \frac{1}{N} \sum_{i=1}^N T_i \quad (4)$$

Além disso, foram avaliados os tempos de treinamento que informam quanto tempo levou para obter um modelo treinado e pronto. Outra métrica foi a perda, que

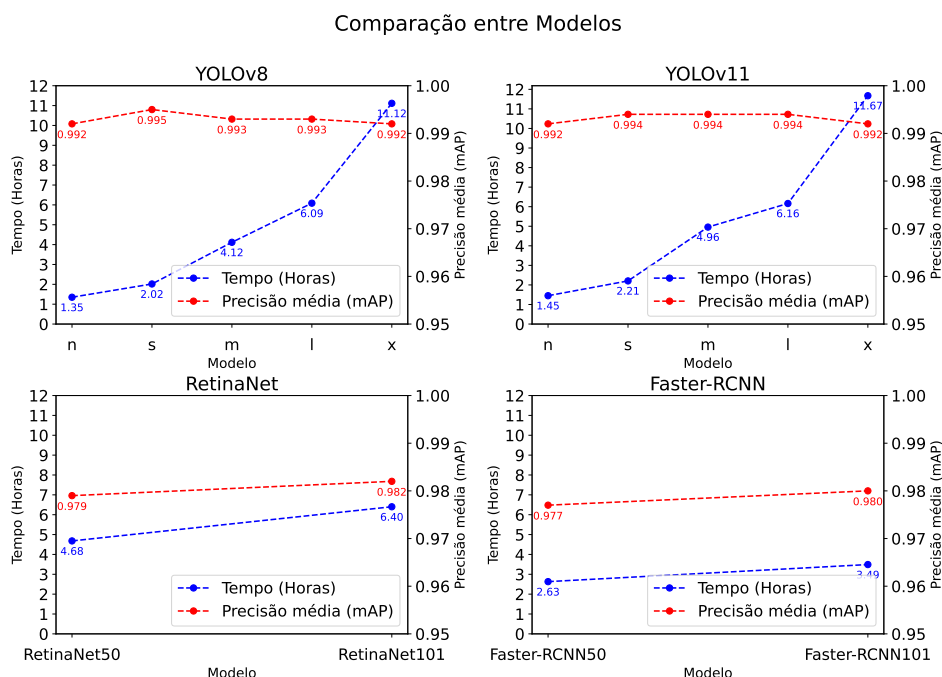


Figura 4. Comparação entre modelos

indica a diferença entre as previsões do modelo e os valores reais durante o treinamento, sendo fundamental para verificar a convergência dos modelos. Por fim, foram registradas as métricas relacionadas aos recursos computacionais gastos. Como ambiente de execução, foi escolhida a plataforma Kaggle, que disponibiliza uma máquina em nuvem composta por um processador Intel Xeon 2GHz, 31 GB de memória RAM e uma placa de vídeo Tesla P100 com 16 GB de VRAM. Já para o treinamento dos modelos, foram utilizadas 100 epochs, com 12 workers e 16 imagens por batch (lote).

4.2. Algoritmos avaliados e avaliações interalgoritmos

Os algoritmos avaliados são variações de versões de YOLO, chamados de YOLO V8 e YOLO V11. Todos eles apresentam algoritmos parecidos, mas distintos apenas em versões e finalidades. Além disso, foram avaliados o Retinanet e Faster R-CNN. Devido à existência de diversas versões e modelos do YOLO, foi necessário selecionar as versões mais relevantes para realizar uma comparação entre os modelos. Para garantir uma avaliação justa, foram utilizadas as mesmas configurações de hardware e treinamento. A versão 8 do YOLO foi escolhida para a comparação por se tratar de uma versão amplamente utilizada pela comunidade devido a sua grande facilidade de uso, documentação abrangente e estabilidade. A versão 11 do YOLO foi selecionada por ser a atualização mais recente até o momento da realização destes testes. Os mesmos procedimentos adotados no treinamento dos modelos da versão 8 foram aplicados para garantir uma comparação justa. Para determinar o modelo com o melhor custo-benefício dentro dessa versão, foram realizados treinamentos em condições iguais, comparado o tempo de treinamento e a precisão média(mAP) obtida por cada modelo.

Figura 4a mostra a avaliação entre diferentes versões do modelo Yolov8 (eixo x) contra o tempo de treinamento e contra o mAP. O modelo que obteve o melhor custo-

benefício foi o YOLOv8n, pois conseguiu obter a melhor precisão média com um menor tempo de treinamento em comparação aos outras versões. Portanto, essa versão foi escolhida para as comparações subsequentes com outros algoritmos de detecção de objetos. Semelhante ao YOLOv8, o modelo que obteve o melhor custo benefício continuou sendo o YOLOv11n, conforme visto na Figura 4b. Sendo assim, essa versão também foi escolhida para as comparações subsequentes com outros algoritmos de detecção de objetos.

O RetinaNet está disponível em duas versões no repositório Detectron2, diferenciando-se principalmente pelo backbone utilizado, onde a primeira versão emprega o algoritmo ResNet-50, enquanto a segunda utiliza a ResNet-101. A principal distinção entre esses algoritmos consiste na profundidade da rede, sendo a ResNet-101 uma variante mais profunda, o que potencialmente melhora a capacidade de extração de características, porém com um custo computacional mais elevado. Como mostrado na Figura 4c, o modelo que obteve o melhor custo-benefício foi o RetinaNet50. Portanto, essa versão foi escolhida para as comparações subsequentes com os demais algoritmos de detecção de objetos.

Faster R-CNN está disponível em várias versões no repositório Detectron2, diferenciando-se principalmente pelo backbone utilizado, que aqui foi utilizado o Feature Pyramid Network (FPN). O treinamento foi realizado utilizando as variantes com os backbones ResNet-50 e ResNet-101. Como podemos observar na Figura 4d, o modelo com backbone ResNet-50 demonstrou um melhor equilíbrio entre desempenho e custo computacional em comparação à versão com ResNet-101. Dessa forma, esse modelo foi selecionado para as próximas comparações entre os demais algoritmos de detecção.

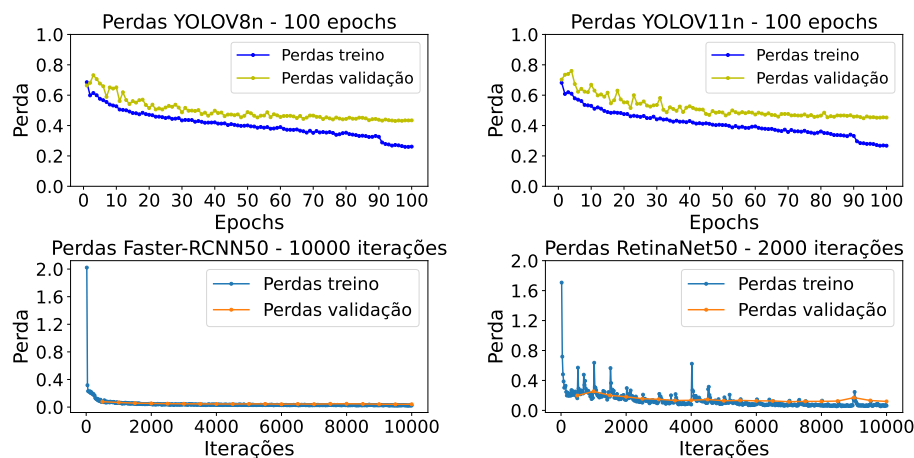


Figura 5. Comparação entre perda de validação e total

4.3. Refinamento dos modelos

Para determinar a eficiência dos modelos treinados, faz-se necessários analisar o comportamento dos mesmos durante o processo de treinamento, observando métricas como perda total e a perda de validação. Observado a Figura 5, pode-se concluir que os modelos YOLOv8 e YOLOv11 a partir da epoch 40 não apresentam uma grande redução no valor de perda, indicando que os modelos já aprenderam as principais características dos dados, já os modelos RetinaNet e Faster-RCNN apresentam esse comportamento a partir de 2000 iterações. Esse comportamento sugere que o número de epochs e iterações podem ser

reduzidos sem comprometer o desempenho dos modelos, resultando em um menor tempo de treinamento e economia de recursos computacionais.

A Tabela 1 demonstra que os modelos treinados com 40 epochs e 2000 iterações apresentam um melhor custo benefício em relação aos modelos que foram treinados com 100 epochs e 10000 iterações. Sendo assim, os mesmos prosseguirão para futuras comparações.

Modelo	Epochs/Iterações	Tempo de Treinamento	mAP:50
YOLOv8	100	1:21:04	0.992
YOLOv11	100	1:26:56	0.995
RetinaNet50	10000	4:40:43	0.979
Faster R-CNN50	10000	2:37:48	0.977
YOLOv8	40	0:30:08	0.992
YOLOv11	40	0:34:48	0.992
RetinaNet50	2000	1:15:36	0.936
Faster R-CNN50	2000	0:42:14	0.941

Tabela 1. Resultados do refinamento (*fine tuning*) dos modelos

Os recursos computacionais utilizados durante o treinamento dos modelos de detecção é de extrema importância para determinar a viabilidade e eficiência de sua implementação. Os modelos YOLOv8 e YOLOv11 apresentaram o melhor custo-benefício ao considerar tanto o consumo computacional quanto o desempenho em termos de mAP50:95 alcançado.

Modelo	Batch	Workers	Disco	RAM	Memó. GPU	mAP50:95
YOLOv8	16	12	2.4	5.1	2.8	0.911
YOLOv11	16	12	2.4	4.9	3.1	0.918
RetinaNet50	8	4	2.6	2.1	8.8	0.786
Faster R-CNN50	8	4	2.6	2.1	8.4	0.757

Tabela 2. Recursos computacionais.

Uma das principais métricas para realizar a avaliação de modelos de detecção de objetos é a Mean Average Precision (mAP), pois permite medir a precisão da predição em relação às anotações reais do conjunto de teste. A Figura 6 apresenta a comparação dos valores de mAP50 para os modelos treinados, onde se observa que os modelos YOLOv8 e YOLOv11 apresentaram melhores valores de mAP gerais, atingindo o valor de 0.992 ambos. Já os modelos RetinaNet e Faster-RCNN obtiveram valores de 0.936 e 0.941, o que representa uma ligeira inferioridade em relação aos modelos YOLO. Porém, analisando o desempenho individual por classe, é possível notar que algumas categorias apresentaram diferenças mais notáveis, como nas classes Meisje met de parel e Mona Lisa. Os modelos YOLO mantiveram um mAP superior a 0.9, enquanto o RetinaNet e o Faster-RCNN apresentaram quedas para valores abaixo de 0.8 em algumas dessas classes.

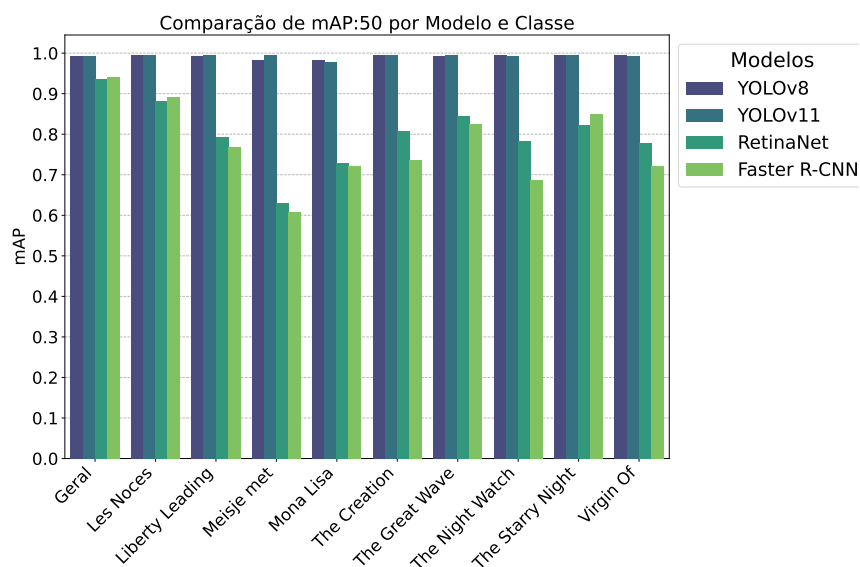


Figura 6. Comparação de mAP50

Figura 7 avalia os valores de recall, onde observa-se que os modelos YOLOv8 e YOLOv11 apresentaram os maiores valores de recall, ambos atingindo o valor de 0.987, indicando um bom desempenho na identificação dos objetos desejados. Já os modelos RetinaNet e Faster-RCNN apresentaram um desempenho razoável, porem com valores inferiores as outros dois modelos anteriores, atingindo os valores de 0.827 e 0.808, respectivamente. Esses valores menores sugerem uma maior propensão a falsos negativos, ou seja, a não identificação de alguns objetos presentes nas imagens.

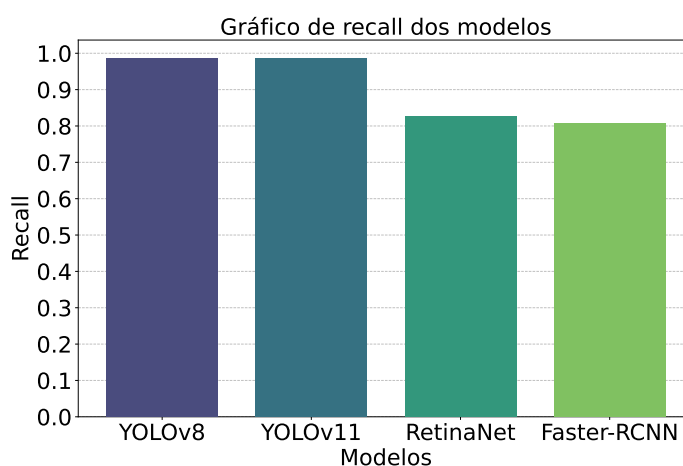


Figura 7. Comparação de Recall

Um aspecto crítico na escolha de um modelo para implementação em projetos práticos, especialmente aqueles que envolvem processamento em tempo real, é o tempo de inferência, ou seja, o tempo que o modelo leva para processar uma imagem ou, em casos de vídeo, cada quadro (frame). Para avaliar essa métrica, foi conduzido um experimento de tempo de inferência utilizando o conjunto de teste do dataset. Durante o experimento, as imagens foram submetidas a um filtro de redimensionamento para analisar o

desempenho dos modelos em diferentes resoluções. As resoluções selecionadas para os testes foram 640x640, 800x800, 1080x1080 e 1280x1280, permitindo uma avaliação do comportamento dos modelos tanto em imagens menores, que exigem menos poder computacional, quanto em imagens maiores, que apresentam maior complexidade e exigem maior capacidade de processamento.

Como mostrado na Figura 8, os modelos YOLO apresentaram o menor tempo de inferência em todas as resoluções testadas, destacando-se especialmente nas resoluções menores. O modelo YOLOv8n, obteve o tempo de inferência mais baixo entre todos os modelos avaliados, esse comportamento pode ser explicado pelo algoritmo otimizado e pela abordagem de detecção mais eficiente do YOLO, que prioriza velocidade sem comprometer significativamente a precisão. Por outro lado, os modelos RetinaNet e Faster R-CNN, apresentaram tempos de inferência mais elevados, principalmente em resoluções maiores. No entanto, é importante ressaltar que, na resolução de 800x800, esses modelos demonstraram tempos de inferência mais baixos em comparação com as resoluções superiores, o que pode ser explicado pelo fato de essa resolução ser mais próxima da resolução nativa desses algoritmos, oferecendo um equilíbrio entre qualidade de detecção e eficiência computacional.

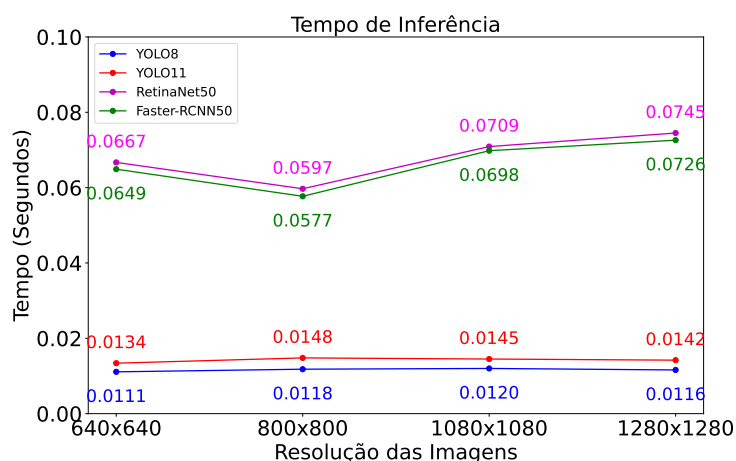


Figura 8. Comparação de tempo de inferência

5. Conclusão

Nesse trabalho, foi realizado uma comparação detalhada de diferentes algoritmos de detecção de objetos, focando em suas aplicações na área de acessibilidade para pessoas com deficiência visual. Os modelos analisados foram o YOLOv8, YOLOv11, RetinaNet e Faster R-CNN, que foram avaliados com base em métricas como precisão média (mAP), recall, consumo de recursos computacionais durante o treino e tempo de inferência. Os resultados obtidos demonstraram que os modelos YOLOv8 e YOLOv11 se destacaram pelo alto desempenho nas métricas de precisão média e recall, com um bom custo-benefício em termos de recursos computacionais. Portanto, YOLO pode ser uma escolha vantajosa em aplicações de detecção de objetos em tempo real, como sistemas de acessibilidade para ambientes culturais e artísticos, tais como esta aplicação de detecção de obras de arte em formato de pintura, tornando assim a arte acessível para todos. Para trabalhos futuros pode-se realizar integração desses modelos em uma aplicação móvel, focado na aces-

sibilidade para pessoas com deficiência visual, permitindo que as mesmas interajam de maneira mais inclusiva com obras de arte, fornecendo informações em tempo real sobre as obras presentes.

Referências

- Bernardo, F. T. (2019). Desenvolvimento de uma interface homem-máquina utilizando visão computacional para acessibilidade digital.
- Boesch, G. (2024). The fundamental guide to faster r-cnn. Acesso em: 21 Mar. de 2025.
- de Carvalho, L. E. S. and do Nascimento, H. A. D. (2025). Humanidades digitais: performatividades na cultura digital. In Fernandes, A., editor, *Os inventores das obras-primas científicas: uma viagem através dos tempos e das tecnologias*, chapter 5, pages 37–49. Publicação CIAR, UFG, Goiânia, Brasil.
- de Milano, D. and Honorato, L. B. (2010). Visão computacional. *Faculdade de Tecnologia, Universidade Estadual de Campinas*.
- IBGE (2023). Pessoas com deficiência têm menor acesso à educação, ao trabalho e à renda. Acesso em: 16 mar. 2025.
- Khan, F., Beigpour, S., Weijer, J., and Felsberg, M. (2014). Painting-91: A large scale database for computational painting categorization. *Machine Vision and Applications*, 25:1385–1397.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988.
- Martins, P. I. S. R. (2008). A inclusão pela arte: museus e públicos com deficiência visual. Master's thesis, Universidade de Lisboa (Portugal).
- Olivatto, T. F. (2021). Identificação automática de rampas de acessibilidade apoiada por visão computacional a partir de imagens panorâmicas street-level.
- Patel, P., Pampaniya, S., Ghosh, A., Raj, R., Karuppai, D., and Kandasamy, S. (2025). Enhancing accessibility through machine learning: A review on visual and hearing impairment technologies. *IEEE Access*, 13:33286–33307.
- Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788.
- Team, A. (2024). Albuementations documentation. Acesso em: 24 de Nov. de 2024.
- Tian, Y., Yang, X., and Arditi, A. (2010). Computer vision-based door detection for accessibility of unfamiliar environments to blind persons. In *Computers Helping People with Special Needs: 12th International Conference, ICCHP 2010, Vienna, Austria, July14-16, 2010, Proceedings, Part II 12*, pages 263–270. Springer.
- Tzutalin and Contributors (2024). Labelimg: A graphical image annotation tool. Acesso em: 11 de Nov. de 2024.